Kubron: анализ проекта и план улучшений

Репозиторий: VadimIl224/kubron (Wails + Go + Vue). Дата анализа: 27.10.2026.

Анализ статический: проект не собирался и тесты не запускались. Объём: около 32 тыс. строк Go (197 файлов, 64 с тестами) и 140 Vue-компонентов. Сильные стороны: архитектура «одно ядро, две оболочки», ADR, DCO, проверка лицензий, allowlist для RPC, e2e на kind.

Высокий приоритет

1. Имя Go-модуля `analog`

Репозиторий называется kubron, а модуль `analog`. Это мешает публичному SDK `pkg/ext`: внешний (платный SSO) модуль не сможет импортировать `analog/pkg/ext` без `replace`. Нужно переименовать в `github.com/VadimIl224/kubron`. Это механическая замена импортов, и чем раньше, тем дешевле.

2. Нет линтеров и форматтеров

- В Go CI делает только `gofmt` и `go vet`. Не хватает `golangci-lint`, `govulncheck` и `go test -race` (на Linux CI cgo есть, так что `-race` работает).
- На фронтенде нет ESLint и Prettier, а `any` встречается 464 раза.
- Нет `.editorconfig` и Makefile/Taskfile с едиными командами `dev`, `test`, `lint`, `build`.

3. `context.Background()` вместо проброса контекста

В `internal/k8s` 92 вызова с зашитыми таймаутами, а контекст принимают только 73 функции. Для десктопа это терпимо, но в веб-режиме и на этапе 2 (многопользовательский режим) запрос не отменится при отключении клиента или выходе из сессии. Нужно пробрасывать `ctx` из методов `App` вниз, а таймауты задавать на верхнем уровне.

4. Логирование

`log/slog` не используется совсем, ошибки выводятся через `println` (`main.go`, `app.go`). Нужны структурированные логи: уровень, request-id и session-id в веб-режиме, вывод в файл для десктопа. Журнал аудита при этом остаётся

отдельным.

5. `internal/app/app.go`: 1329 строк и 152 метода

Это god-object. Его стоит разбить по доменам (clusters, workloads, helm, argocd, auth) на небольшие сервисы со своими файлами.

Вызов `a.authz(...)` повторяется 150 раз, и защищает от забытой проверки только тест `policy_test`. Лучше обернуть вызов в хелпер или middleware.

Сейчас часть методов при отказе авторизации молча возвращает `nil` (например, `GetClusterStatuses`), и UI показывает пустой список вместо ошибки доступа. Здесь нужно вернуть `error`.

Средний приоритет

Фронтенд

- `types/index.ts` (765 строк) дублирует Go-структуры вручную. Лучше брать типы из Wails-моделей (`wailsjs/go/models`) или генерировать через `tygo`, иначе они разойдутся.
- Нужен `vue-router`. Для веб-версии необходимы ссылки на вид или ресурс и работающая кнопка «назад»; сейчас навигация идёт через `useViewRegistry`.
- 22 синглтона `ref` на уровне модулей в composables работают, но неудобны для тестов и devtools. Для общего состояния (кластеры, сессия, namespace) подойдёт Pinia.
- `xterm` и `xterm-addon-fit` — устаревшие пакеты, их заменили `@xterm/xterm` и `@xterm/addon-fit`.
- Тесты запускаются только в `environment: 'node'`, при этом 140 компонентов покрыты 21 тестовым файлом. Стоит добавить `@vue/test-utils` и `happy-dom` для компонентных тестов.
- В `tsconfig` нет `noUncheckedIndexedAccess`, а `tsconfig.node.json` использует `moduleResolution: node` с `ignoreDeprecations: "6.0"`. Правильнее `bundler`. Файл `tsconfig.node.tsbuildinfo` лежит в репозитории, его нужно добавить в `.gitignore`.
- В `vite.config.ts` два похожих плагина-заглушки для wailsjs. Их можно заменить одним механизмом на основе alias.
- 87 `.vue`-файлов содержат захардкоженный русский текст. Если проект открытый и международный, нужен `vue-i18n`; если нет, это можно оставить.

CI и релизы

- Нет релизного пайплайна: сборки Wails под Windows, macOS и Linux, подписанные артефакты, Docker-образ `kubron-server` (multi-stage, distroless, non-root).
- E2E стоит `continue-on-error`. Как только он стабилен, его нужно сделать блокирующим.
- В CI Go-версия `stable`, а в `go.mod` указана конкретная. Лучше `go-version-file: go.mod`.
- Шаги apt и Wails CLI повторяются в трёх джобах, их можно вынести в composite action.
- Нужны `concurrency: cancel-in-progress` и закрепление actions по SHA.
- Версия сейчас в трёх местах: `1.0.0` в `wails.json`, `0.0.0` в `package.json` и ничего в Go. Нужен один источник, например тег плюс `-ldflags`.

Безопасность (веб-режим)

- Сессии, CSRF и проверка Origin и Fetch-Metadata сделаны хорошо. При этом я не нашёл в `internal/server` ограничения попыток входа, security-заголовков (CSP, X-Content-Type-Options) и `IdleTimeout`. Возможно, защита от перебора есть в `auth.Service`, это стоит проверить.
- Kubeconfig, токены Argo и пароли Helm хранятся обычными файлами с правами 0600. Для десктопа можно использовать системное хранилище ключей (Keychain, Credential Manager, libsecret), для сервера — шифрование на уровне приложения (по плану это ADR-0008).

Низкий приоритет

- README скудный: нет скриншотов, инструкции по `wails dev` и сборке десктопа, схемы архитектуры.
- Нет CHANGELOG.
- В `wails.json` пустой email автора.
- Переход на Wails v3 пока не стоит делать: он ещё не стабилен.

С чего начать

Четыре быстрых изменения, каждое отдельным PR:

1. Переименование модуля.
2. golangci-lint, ESLint, Prettier и `.editorconfig` с правками CI.

3. Переход на `slog`.
4. Замена xterm-пакетов.

После этого можно переходить к проброске `ctx` и разбиению `app.go`: это крупные рефакторинги, и им нужна страховка из линтеров и тестов.

Go-код может не собираться в окружении ассистента, поэтому после его правок нужны `go test` и `wails build` у владельца репозитория.